

Zephyr host mocks (mocks/)

Native Linux simulation of the handshake IMU pipeline for race hunting,
memory-model checks, and desk-free iteration. No ESP32 board, no Renode, no QEMU.

Binary: `mocks/build/handshake_host`

Sanitizer sweep: `mocks/sanitize.sh`

Contents

- [Why host mocks](#)
 - [Quick start](#)
 - [CLI parameters](#)
 - [Environment variables](#)
 - [What is compiled](#)
 - [Thread model](#)
 - [Memory model \(ESP32-S3 flat arena\)](#)
 - [Optional SDL RGB565](#)
 - [BLE / NVS stubs](#)
 - [Host logger](#)
 - [Sanitizers](#)
 - [Valgrind](#)
 - [CMake options](#)
 - [Examples](#)
 - [CI notes \(future\)](#)
 - [Limitations](#)
-

Why host mocks

Desk soft-reset / IMU-recover hangs are hard to bisect on-device. The host binary

links **real** `imu_pipeline.c` (+ `cal` / `attitude` / `walk`) against pthread stand-ins

for Zephyr so **ThreadSanitizer** can see true data races.

Approach	Good for	Weak for
Desk USB flash	real PSRAM / BLE / panel	slow loop, silent hangs
Renode / QEMU xtensa	ISA-level	ESP32-S3 BLE+PSRAM fidelity
Host + TSan	app races, recover stress	not a CPU/PLL model

Quick start

```
cd mocks

./build.sh --seconds 8

./build.sh --seconds 6 --stress-recover

./sanitize.sh
```

build.sh configures Clang + TSan + optional SDL, builds, and runs with your args.

CLI parameters

Flag	Default	Meaning						
--seconds N	8	Wall / virtual run duration						
--virtual-time	off	Advance clock in software (no sleep)						
--stress-recover	off	Request IMU recover every 1.5s after warm-up						
--log-level LVL	inf	`err\	wrn\	inf\	dbg\	trace\	func\	all or 0..10`
--nvs-path PATH	mocks_nvs.bin	File-backed NVS / crash-ring store						
--no-sdl	—	Force framebuffer-only						
--sdl	want on	Request SDL if built + display present						
--no-flat-heap	—	Use libc malloc (also set env for wrap builds)						
--flat-heap	want on	Force ESP32 DRAM+PSRAM arena						
-h / --help	—	Usage						

Exit codes (normal mode, not stress):

Code	Meaning
0	IMU live + BLE script reached NOTIFY
1	Bad args / init failure

Code	Meaning
2	IMU never became live
3	BLE script never reached NOTIFY

`--stress-recover` always exits `0` if the process survives (mid-recover is expected).

Environment variables

Variable	Effect
<code>HOST_NO_SDL=1</code>	Skip SDL even if linked (headless / CI)
<code>HOST_NO_FLAT_HEAP=1</code>	Disable arena before first alloc (needed when libc is <code>--wrap ped</code>)
<code>TSAN_OPTIONS</code>	Passed through (default <code>halt_on_error=1:...</code>)
<code>ASAN_OPTIONS</code>	Default <code>detect_leaks=1:halt_on_error=1</code>
<code>UBSAN_OPTIONS</code>	Default <code>halt_on_error=1:print_stacktrace=1</code>
<code>HOST_SAN_SECONDS</code>	Duration for <code>sanitize.sh</code> non-Valgrind runs
<code>HOST_VALGRIND_SECONDS</code>	Duration for Valgrind run (default <code>12</code>)
<code>CC</code>	Compiler for cmake (default <code>clang</code>)
<code>DISPLAY</code> / <code>WAYLAND_DISPLAY</code>	Absence $\Rightarrow$ SDL auto-skip

What is compiled

Firmware unit	Host
<code>zephyr/app/handshake/src/imu_pipeline.c</code>	<code>as-is</code>
<code>imu_cal.c</code> , <code>attitude.c</code> , <code>walk_distance.c</code>	<code>as-is</code>
<code>qmi8658.c</code>	<code>src/qmi8658_host.c</code> (synthetic still + noise)
<code>k_mutex</code> / <code>workqueue</code> / <code>sleep</code> / <code>uptime</code>	<code>src/kernel_host.c</code>
<code>power</code> / <code>floor</code> / <code>vibro</code> / <code>zoom</code> / <code>stall WDT</code>	<code>src/stubs_host.c</code>
<code>panel RGB565</code>	<code>src/host_fb.c</code> (+ optional SDL)
<code>BLE NOTIFY path</code>	<code>src/ble_host.c</code>
<code>NVS</code> / <code>crash ring</code>	<code>src/host_nvs.c</code>
<code>logger</code>	<code>src/host_debug.c</code>
<code>heap</code>	<code>src/host_mem.c</code> (+ optional <code>host_mem_wrap.c</code>)

`main_host` shape mirrors the desk freeze: IMU workqueue recover + main poll +

render snapshot ~30 Hz + BLE drain.

Thread model

```
main          imu_pipeline_poll / heartbeat / optional stress recover

imu_wq        recover_work_fn (calibrate, set ready)

render        imu_pipeline_snapshot + paint RGB565 + BLE enqueue

ble_loop      drain NOTIFY_FIFO + scripted connect/NOTIFY
```

Status flags `g_ready` / `g_live` / `g_recovering` are Zephyr `atomic_t` so poll and recover do not race on plain `bool` (TSan caught this under `--stress-recover`).

Memory model (ESP32-S3 flat arena)

Desk board (Waveshare ESP32-S3 LCD 1.47"):

- Internal SRAM $\approx$ **512 KiB**
- Octal PSRAM **8 MiB** (`CONFIG_ESP_SPIRAM_SIZE`)
- System heap pool 64 KiB (`CONFIG_HEAP_MEM_POOL_SIZE`) — firmware still uses

`k_malloc` / `heap_caps_*` on device; host approximates with one arena

Host arena layout

At `host_mem_init()` the simulator `mmap`s one contiguous region:

```
offset 0                +512 KiB                +8 MiB

|<----- DRAM ----->|<----- PSRAM ----->|
```

Total $\approx$ **8.5 MiB**. All `k_malloc` / `k_free` and (when wrap is on) libc

`malloc` / `calloc` / `realloc` / `free` carve **blocks from this mapping**.

Allocator behaviour

- **Alignment:** 8-byte payloads (Xtensa / ESP-IDF style)
- **Redzone:** 8 bytes after each payload (`0xA5`); checked on free/realloc
- **Poison:** new payload `0xCD` , freed payload `0xDD`
- **Placement:** size ≤ 4096 prefers DRAM region; larger prefers PSRAM
- **Faults (abort):** misaligned free, double-free, bad magic, smashed redzone
- **Stats:** logged at exit (`host_mem_log_stats`)

Why this helps

glibc's heap hides cross-object overflows and “wrong” alignments that matter on

ESP32. A fixed flat arena:

1. Keeps all sim heap traffic in one address range
2. Surfaces **write-past-end** via redzones
3. Surfaces **misaligned** free/realloc
4. Makes “ran out of DRAM” / fragmentation behaviour closer to a capped board

It is **not** a cycle-accurate SPIRAM cache model and does not emulate DMA

capability rules (`esp_ptr_dma_capable`).

Sanitizer interaction

Build	Flat arena	libc --wrap
TSan / plain	ON	ON (default)
ASan	ON (<code>k_malloc</code>)	OFF (ASan owns malloc)
Valgrind run	OFF via <code>HOST_NO_FLAT_HEAP=1</code>	wrap present but disabled

Optional SDL RGB565

- **Build:** `pkg-config sdl2` → `-DHOST_HAS_SDL2`
- **Runtime auto-skip:** no `DISPLAY` / `WAYLAND_DISPLAY` , or `HOST_NO_SDL=1` , or `--no-sdl`
- Soft-fails if `SDL_Init` fails — never fails the sim
- Safe for headless builders: do **not** require Xvfb for other mock steps

Framebuffer size: **172×320** RGB565 (`HOST_FB_W` / `HOST_FB_H`).

BLE / NVS stubs

BLE (`ble_host`): scripted phone — connect @ 800 ms, NOTIFY @ 1500 ms (armed from `main`). Render enqueues samples into a drop-oldest FIFO; `ble_loop` drains them.

NVS (`host_nvs`): single file blob table + crash-ring keys `crash.count` / `crash.N` . Path via `--nvs-path` .

Host logger

Format inspired by `copy_recovery` `debug.h` :

```
[INF 123 tid=12345 core=2 name=main] <imu_pipe recover_work_fn imu_pipeline.c:338> IMU recover OK
```

Zephyr `LOG_*` macros map here. Also `HOST_LOG`, `P_DBG`, `LOG_API_ENTER/LEAVE`.

Sanitizers

```
cd mocks

./sanitize.sh
```

Runs, in order:

1. **TSan** — data races / deadlocks
2. **ASan + UBSan** — heap/stack OOB, UB
3. **plain** — no sanitizer (Valgrind input)
4. **Valgrind** (if installed) — definite leaks only
5. **TSan --stress-recover** — recover races under load

Example output markers: `OK tsan`, `OK asan_ubsan`, `OK plain`, `OK valgrind`,
`OK tsan-stress`.

Re-run one config:

```
cmake -S . -B build-tsan -DCMAKE_C_COMPILER=clang -DHOST_TSAN=ON

cmake --build build-tsan -j$(nproc)

HOST_NO_SDL=1 ./build-tsan/handshake_host --seconds 4 --stress-recover
```

Valgrind

Laptop: Valgrind 3.x available. CI image: optional.

```
HOST_NO_FLAT_HEAP=1 HOST_NO_SDL=1 \  
  
valgrind --leak-check=full --errors-for-leak-kinds=definite --error-exitcode=99 \  
  
./build-plain/handshake_host --seconds 12 --no-flat-heap
```

Notes:

- Sanitizer builds + Valgrind = do not mix
- Workqueue pthread TLS often shows as **possibly lost** — ignored by `sanitize.sh`
- Flat arena bypasses Memcheck's heap tracker → disable it for Valgrind runs

CMake options

Option	Default	Notes
HOST_TSAN	ON	Mutually exclusive with ASan
HOST_ASAN	OFF	
HOST_UBSAN	OFF	Often paired with ASan
HOST_ENABLE_SDL	ON	Soft if sdl2 missing
HOST_FLAT_HEAP	ON	Map arena
HOST_FLAT_HEAP_WRAP	ON	Forced OFF when ASan ON

Examples

Smoke (TSan, headless)

```
HOST_NO_SDL=1 ./build.sh --seconds 5 --log-level inf
```

Chatty mutex / recover

```
./build.sh --seconds 4 --log-level trace --stress-recover
```

SDL window (local desktop)

```
# DISPLAY already set on Plasma/Wayland+XWayland

./build/handshake_host --seconds 30 --sdl --log-level wrn
```

Force libc heap

```
HOST_NO_FLAT_HEAP=1 ./build/handshake_host --seconds 3 --no-flat-heap
```

Virtual time (fast forward)

```
./build/handshake_host --seconds 20 --virtual-time --log-level inf
```

Persist NVS across runs

```
./build/handshake_host --seconds 3 --nvs-path /tmp/hs_nvs.bin

./build/handshake_host --seconds 3 --nvs-path /tmp/hs_nvs.bin # crash_ring grows
```

CI notes (future)

When adding a pipeline job:

1. Call `./mocks/sanitize.sh` (sets `HOST_NO_SDL=1`)
2. Do **not** require SDL / Xvfb for green builds
3. Valgrind optional (`SKIP` if missing)
4. Flat-heap wrap OK under TSan; ASan keeps wrap off

Limitations

- No Wi-Fi / BT controller / PSRAM cache timing
- No panel SPI / DMA bounce path
- QMI8658 is synthetic — not I²C bit-exact
- Soft-reset PSRAM hang on desk may still need on-device capture
- Early CRT `malloc` before `main` sees default flat-heap-on; use

`HOST_NO_FLAT_HEAP=1` to force libc for the whole process

Related docs

- `mocks/README.md` — short cheat sheet
- `docs/zephyr-build.md` — west / flash
- `docs/architecture.md` — product overview